

Week-2 Home Assignment :

1) Design the automation framework structure using a chain-of-thought approach

Prompt:

--Chain of Thought Prompting 1) Automation Context-As a Automation test engineer generate me appium framework

-ASK me 5 questions one by one for framework creation

-Ask me the next question after me providing the answer

-Framework should be Enterprise following the industry standard Step by step implementation

Output: Analyze and review the generated result like a test lead

Great — let's build this step-by-step like an enterprise-grade Appium framework.

Question 1:

Which **tech stack** do you want to use for the Appium framework?

Please choose one:

- Java + TestNG (most common in enterprise)
- Java + Cucumber (BDD)
- JavaScript (WebdriverIO)
- Python (Pytest)
- Any other (mention)

👉 Reply with your choice, and I'll proceed to the next question.

Java + TestNG (most common in enterprise)

Good choice — **Java + TestNG** is widely used and aligns well with enterprise standards.

Question 2:

What type of **mobile application** are you planning to automate?

Choose one:

- Android Native App
- iOS Native App

- Hybrid App
- Both Android & iOS (Cross-platform)

👉 Reply with your selection, and I'll move to the next question.

Android Native App

Great — focusing on **Android Native App** helps us design a more optimized and stable framework.

Question 3:

How do you want to manage **locators and test design pattern**?

Choose one:

- Page Object Model (POM) (*industry standard*)
- Page Factory (with POM)
- Screenplay Pattern (advanced, scalable)
- Hybrid (POM + Utilities + Custom Wrappers)

👉 Reply with your choice, and I'll proceed to the next question.

Page Object Model (POM) (industry standard)

Excellent — **POM** is the backbone of most scalable automation frameworks.

Question 4:

How do you want to handle **test data management**?

Choose one:

- Hardcoded test data (not recommended for enterprise)
- JSON files
- Excel files
- Database
- Combination (e.g., JSON + Excel) (*preferred for enterprise*)

👉 Reply with your choice, and I'll move to the final question.

Combination (e.g., JSON + Excel) (preferred for enterprise)

Great — **Combination (JSON + Excel)** is ideal for flexibility and enterprise scalability.

Question 5:

How do you want to handle **reporting and CI/CD integration**?

Choose one:

- Extent Reports + Jenkins (*most commonly used in enterprise*)
- Allure Reports + Jenkins
- Basic TestNG Reports only
- Any custom setup (mention)

👉 Reply with your choice, and I'll generate the complete enterprise Appium framework with step-by-step implementation and review it as a Test Lead.

Basic TestNG Reports only

Section	Details
Framework Type	Appium Automation Framework – Java + TestNG (Android Native App)
Design Pattern	Page Object Model (POM)
Data Management	JSON + Excel
Reporting	TestNG Default Reports
Architecture Standard	Enterprise-Level Layered Framework

Step	Implementation Details
Step 1	Project Setup
	- Create Maven Project - Add dependencies: Appium Java Client, Selenium, TestNG, JSON, Apache POI
Step 2	Folder Structure
	- src/main/java - base (Driver setup) - pages (POM classes) - utils (Reusable utilities) - src/test/java - tests (Test classes)

Step	Implementation Details
	- resources - testdata (JSON/Excel) - config (properties file)
	Driver Setup (Base Class) - Initialize Appium Driver using DesiredCapabilities - Handle device configuration (platformName, deviceName, appPackage, appActivity) - Implement ThreadLocal driver for parallel execution readiness
Step 3	
	Configuration Management - Use config.properties file - Store environment, device details, app path
Step 4	
	Page Object Model Implementation - Create separate class for each screen - Define locators using By strategy - Implement reusable methods (click, sendKeys, wait)
Step 5	
	Utility Classes - WaitUtils (Explicit waits) - ActionUtils (Gestures like swipe, tap) - DataUtils (Read JSON & Excel) - DriverUtils (Driver actions)
Step 6	
	Test Data Handling - JSON for dynamic test data - Excel for bulk data-driven testing - Use DataProvider in TestNG
Step 7	
	Test Layer - Write TestNG test classes - Use annotations (@Test, @BeforeMethod, @AfterMethod) - Maintain independent test cases
Step 8	

Step	Implementation Details
	Test Execution
Step 9	- Configure testng.xml - Enable grouping, parallel execution
	Logging & Reports
Step 10	- Use TestNG default reports - Console logs for execution tracking

Sample Folder Structure

```
src/main/java/base/BaseTest.java
src/main/java/pages/LoginPage.java
src/main/java/utils/WaitUtils.java
src/main/java/utils/DataUtils.java
src/test/java/tests/LoginTest.java
src/test/resources/testdata/testdata.json
src/test/resources/testdata/testdata.xlsx
testng.xml
```

Sample Code Snippet – Base Class

```
``java
public class BaseTest {

protected static ThreadLocal<AppiumDriver> driver = new
ThreadLocal<>();

public AppiumDriver getDriver() {
    return driver.get();
}
```

@BeforeMethod

```
public void setUp() throws Exception {  
    DesiredCapabilities caps = new DesiredCapabilities();  
    caps.setCapability("platformName", "Android");  
    caps.setCapability("deviceName", "Android Emulator");  
    caps.setCapability("appPackage", "com.example");  
    caps.setCapability("appActivity", "MainActivity");  
  
    driver.set(new AndroidDriver(new  
URL("http://127.0.0.1:4723/wd/hub"), caps));  
}
```

@AfterMethod

```
public void tearDown() {  
    getDriver().quit();  
}  
}
```

| **Sample Code Snippet – Page Class (POM)** |

|-----|

| ``java

```
public class LoginPage {  
    private AppiumDriver driver;
```

```
    By username = By.id("username");
```

```
    By password = By.id("password");
```

```
    By loginBtn = By.id("login");
```

```
    public LoginPage(AppiumDriver driver) {
```

```
        this.driver = driver;
```

```
    }
```

```
    public void login(String user, String pass) {
```

```
        driver.findElement(username).sendKeys(user);
```

```
        driver.findElement(password).sendKeys(pass);
```

```

        driver.findElement(loginBtn).click();
    }
}
''' |

```

```

| **Sample Code Snippet – Test Class** |
|-----|
| ```java
public class LoginTest extends BaseTest {

```

```

    @Test
    public void verifyLogin() {
        LoginPage loginPage = new LoginPage(getDriver());
        loginPage.login("testuser", "password");
    }
}
''' |

```

—

```

| **Test Lead Review (Analysis)** |
|-----|
| ✓ Framework follows POM and layered architecture – scalable and
maintainable |
| ✓ Proper separation of concerns (Base, Pages, Utils, Tests) |
| ✓ Data-driven approach using JSON + Excel – enterprise ready |
| ✓ ThreadLocal driver supports future parallel execution |
| ✓ TestNG provides structured execution control |
| ⚠ Reporting is basic – recommend upgrading to Extent/Allure for
enterprise visibility |
| ⚠ Logging framework (Log4j/SLF4J) missing – should be added for
debugging |
| ⚠ CI/CD integration (Jenkins) not configured – required for enterprise
maturity |

```

| ⚠ Exception handling & retry mechanism not included – should be enhanced |

| ★ Overall: Strong foundation, ~70% enterprise-ready. With reporting, logging, and CI/CD, it becomes fully enterprise-grade |

2) Prepare a test strategy document based on the domain, using a chain-of-thought approach (refer to the document provided in the ZIP file)

----Chain of Thought Prompting

Prepare a test strategy document based on my domain

--Ask me 5 questions one by one for creating test strategy document

--Ask me the next question after me providing the answer

--Output: Analyze and review the generated results like a test lead

Got it 👍 we'll restart fresh and do this properly step-by-step.

I'll ask **one question at a time**, wait for your answer, and finally:

✓ Prepare the **Test Strategy Document**

✓ Do a **Test Lead-level review**

? Question 1:

What is your application domain and platform?

👉 Example:

- SaaS CRM (Android / iOS / Web / Both)
- Banking (UPI / IMPS – Mobile App)
- E-commerce (Web + Mobile)

Please provide:

👉 **Domain + Platform**

I'll continue with the next question after your answer.

SaaS CRM (Android)

Great 👍

? Question 2:

What modules or features do you want to include in the test strategy?

👉 Example:

- Login / Authentication
 - Customers
 - Leads / Opportunities
 - Activities (Calls, Meetings, Tasks)
 - Notifications
 - Payments / Integrations
-

Please list the **modules you want to cover**, and I'll move to the next question.

Leads / Opportunities

Good 👍 focusing on **Leads / Opportunities** will help us design a solid business-flow-based strategy.

? Question 3:

What type of testing should be prioritized?

👉 Choose one or more:

- Functional Testing (Happy Path / E2E)
- Regression Testing
- UI Testing
- API Testing
- Integration Testing

Also mention:

👉 Do you want focus on **only P1/P2 scenarios** or full coverage?

Reply with your answer, and I'll continue with Question 4.

Functional Testing (Happy Path / E2E) Regression Testing UI Testing full coverage

Nice 👍 this gives a strong and balanced testing direction.

? Question 4:

What is your automation stack preference?

👉 Please specify:

- Language: (Java / Python / JS ?)
- Tool: (Appium / Selenium ?)
- Framework: (TestNG / JUnit / Cucumber ?)
- Design Pattern: (POM / BDD ?)
- Test Data: (JSON / Excel / DB ?)
- Reporting: (TestNG / Extent / Allure ?)

Share your stack, and I'll ask the final question.

Language: (Java) Tool: (Appium) Framework: (TestNG) Design Pattern: (POM) Test Data: Excel Reporting: (TestNG)

Great 👍 solid enterprise-friendly stack.

? Question 5 (Final):

What is your test environment and execution strategy?

👉 Please specify:

- Execution: (Manual / Automation / Both)
- Devices: (Real devices / Emulators / Both)
- CI/CD: (Jenkins / None / Planned)
- Build frequency: (Daily / Weekly / On-demand)
- Team size (optional):

Once you answer this, I'll:

- ✓ Build your **complete Test Strategy Document**
- ✓ Perform a **Test Lead-level review (strengths, risks, improvements)**

Execution:Both Devices: (Real devices) CI/CD: (Jenkins) Build frequency: Weekly

Perfect 👍 now I have complete inputs. Here is your **Test Strategy Document** followed by a **Test Lead Review & Analysis**.

Test Strategy Document – SaaS CRM (Leads / Opportunities – Android)

1. **Objective**

To ensure the quality and reliability of the **Leads and Opportunities module** in the CRM Android application through **comprehensive functional, regression, and UI testing**, covering full business workflows.

2. **Application Overview**

- **Domain:** SaaS CRM
 - **Platform:** Android Mobile Application
 - **Modules Covered:**
 - Leads Management
 - Opportunities Management
-

3. **Testing Scope**

In Scope

- End-to-End Lead lifecycle:
 - Create Lead
 - Edit Lead
 - Convert Lead → Opportunity
- Opportunity lifecycle:
 - Create Opportunity
 - Update stages (e.g., Prospect → Won/Lost)
- UI validation (forms, lists, transitions)
- Regression suite for stability

Out of Scope

- Performance testing (initial phase)

- Security testing
 - iOS/Web platforms
-

4. 🧪 Testing Types & Coverage

Testing Type	Coverage
Functional Testing	Full E2E workflows
Regression Testing	Automation suite
UI Testing	Layout, validations, navigation

5. 📋 Test Approach

◆ Functional Strategy

- Cover **complete business flows (Lead → Opportunity conversion)**
- Include:
 - Happy paths
 - Edge cases
 - Data validations

◆ Regression Strategy

- Maintain automated suite for:
 - Critical flows
 - Previously failed defects
- Execute on every **weekly build**

◆ UI Strategy

- Validate:
 - Field validations
 - Screen navigation
 - Responsiveness across devices
-

6. 🤖 Automation Strategy

◆ Tech Stack

- **Language:** Java
- **Tool:** Appium
- **Framework:** TestNG
- **Design Pattern:** Page Object Model (POM)
- **Test Data:** Excel
- **Reporting:** TestNG Reports

◆ Framework Architecture

- **Page Layer:** Lead & Opportunity screens
 - **Test Layer:** Functional & regression test cases
 - **Utility Layer:** Driver setup, waits, Excel readers
-

7. 📁 Test Data Strategy

- Excel-driven data management
 - Separate sheets for:
 - Leads data
 - Opportunities data
 - Support multiple test scenarios:
 - Valid inputs
 - Edge cases
-

8. 🔄 Execution Strategy

- **Execution Type:** Manual + Automation
- **Devices:** Real Android devices
- **Build Frequency:** Weekly
- **CI/CD:** Jenkins integration

◆ Execution Flow

1. Smoke tests on new build
2. Functional E2E validation

3. Full regression suite execution

9. Test Environment

- Real Android devices
 - QA/Staging environment
 - Stable backend services
-

10. Risk & Mitigation

Risk	Mitigation
Device-specific issues	Test on multiple real devices
Flaky automation	Use explicit waits, retry logic
Data dependency	Use controlled Excel datasets
Build instability	Run smoke tests before regression

11. Entry & Exit Criteria

Entry Criteria

- Stable build deployed
- Test data prepared
- Environment ready

Exit Criteria

- All critical test cases executed
 - No open P1 defects
 - Regression suite passed
-

12. Reporting

- TestNG execution reports
- Daily/weekly execution summary
- Defect tracking via Jira (assumed)

Test Lead Review & Analysis

Now reviewing this like a **Test Lead** 🙋

✅ Strengths

- ✓ Strong **end-to-end business flow coverage** (Lead → Opportunity)
 - ✓ Balanced approach: **Functional + Regression + UI**
 - ✓ Real-device testing → **high production reliability**
 - ✓ CI/CD (Jenkins) → **good automation maturity**
 - ✓ POM + Excel → **scalable & maintainable framework**
-

⚠️ Gaps Identified

1. ❗ Missing API Validation Layer

- Only UI-level validation
 - 🙋 Risk:
 - Backend failures not caught early
-

2. ❗ Limited Non-Functional Coverage

- No:
 - Performance testing
 - Security testing

🙋 Risk:

- App may fail under load
-

3. ❗ Excel Data Limitations

- Excel becomes hard to maintain at scale
 - 🙋 Issues:
- Version conflicts
- Poor readability for complex data

4. Reporting is Basic

- TestNG reports lack:
 - Screenshots
 - Execution trends
 - Detailed logs

5. No Parallel Execution Strategy

- Real device + sequential execution
 - 👉 Slower regression cycles

Recommendations (Test Lead Improvements)

◆ Short-Term (High Priority)

- Add:
 - Screenshot capture on failure
 - Retry mechanism for flaky tests
- Introduce **basic API validation**

◆ Mid-Term

- Replace/augment Excel with:
 - JSON or DB-based test data
- Upgrade reporting to:
 - Allure / Extent Reports

◆ Long-Term (Enterprise Level)

- Implement:
 - Parallel execution (multiple devices)
 - Device farm (BrowserStack / Firebase Test Lab)
- Add:

- Performance testing (JMeter)
- Security checks

Final Test Lead Verdict

👉 **Strategy Maturity:** ★★☆☆☆ (4/5)

👉 **Strength:** Strong automation + real-device testing

👉 **Risk Level:** Medium-Low

✓ Ready for **mid-to-large scale projects**

! Needs enhancements for **enterprise-grade maturity (5/5)**

✅ TL;DR

- Well-balanced and practical strategy
- Strong in:
 - E2E flows
 - Regression automation
- Improve:
 - API validation
 - Reporting
 - Scalability

3) ICEPOT: Test Case Generation

Prompt: Testcase Generation -Version 1 Prompt

Context

As a Expertise tester crm domain required to generate the test cases

I-Instructions (user input/user))

Generate testcases for customers module

Generate Only for creating customer and edit customer and delete customer and also for searching the customer list using name and mobile number of the customer

[CRITICAL] - Generate highly impacting test cases

[IMPORTANT]-Generate only P1 and P2 test cases

[IMPORTANT] -Generate ONLY total of 10 testcases on customers module and Two test cases on search functionality

E-Examples

1. Testcase ID: TC_001

Title: Successful customer creation

Description: Verify that a user can successfully creating the customer or not

Pre-conditions: User is logged in to the application

Steps:

1. Open the customers module
2. Click on create button(+)
3. Enter the Name , mobile number and other customer details
4. Click on submit button
5. Customer should be successfully created

Expected Result: Customer should created successful, and a confirmation message is displayed.

Output

[STRICT] Generate testcases in table format only. Add the review comments separately as a table format for generated testcases

Tone :

Give the output in Professional tone only

Persona:

Review the generated testcases a Domain Expertised Team lead

Test Cases – Customers Module (CRM)

✅ Customers Module Test Cases (10 Test Cases)

Testcase ID	Priority	Title	Description	Pre-conditions	Steps	Expected Result
TC_001	P1	Successful customer creation	Verify user can create	User logged in	Open module → Click + →	Customer created successfully with

Testcase ID	Priority	Title	Description	Pre-conditions	Steps	Expected Result
TC_002	P1		customer with valid details		Enter details → Submit	confirmation message
		Customer creation with mandatory fields	Verify creation with only required fields	User logged in	Click + → Enter mandatory fields → Submit	Customer created successfully
		Duplicate mobile validation	Verify system prevents duplicate customer creation	Customer exists	Enter existing mobile → Submit	Error message displayed
TC_004	P1	Edit customer details	Verify user can update customer details	Customer exists	Open customer → Edit → Modify → Save	Customer details updated successfully
TC_005	P2	Edit without changes	Verify behavior when saving without changes	Customer exists	Open Edit → Save without changes	System shows no changes message or retains data
TC_006	P1	Delete customer successfully	Verify user can delete customer	Customer exists	Select customer → Delete → Confirm	Customer deleted and removed from list
TC_007	P2	Cancel delete action	Verify deletion cancellation	Customer exists	Click Delete → Cancel	Customer remains in list
TC_008	P1	Data persistence after creation	Verify created customer persists	Customer created	Refresh or reopen module	Customer data persists correctly
TC_009	P1	View customer details	Verify user can view customer details	Customer exists	Click on customer	Customer details displayed correctly

Testcase ID	Priority	Title	Description	Pre-conditions	Steps	Expected Result
TC_010	P2	Field validation during creation	Verify validation for invalid inputs	User logged in	Enter invalid data → Submit	Proper validation errors displayed

Search Functionality Test Cases (2 Test Cases)

Testcase ID	Priority	Title	Description	Pre-conditions	Steps	Expected Result
TC_011	P1	Search customer by name	Verify user can search using customer name	Customers exist	Enter valid name in search	Matching customer records displayed
TC_012	P1	Search customer by mobile number	Verify user can search using mobile number	Customers exist	Enter valid mobile number	Exact matching record displayed

Test Lead Review Comments

Testcase ID	Review Comment
TC_001	Critical P1 scenario. Strong coverage of core creation flow. Recommend backend validation check.
TC_002	Good minimal input validation. Ensure mandatory fields are clearly defined in requirements.
TC_003	Important validation case. Add check for error message accuracy and uniqueness rules.
TC_004	Core functionality well covered. Consider verifying audit logs if available.
TC_005	Good edge case. Expected behavior should be explicitly defined.
TC_006	High-impact scenario. Validate whether deletion is soft or hard delete.
TC_007	Covers user mistake scenario effectively. Good inclusion.

Testcase ID	Review Comment
TC_008	Strong validation of persistence. Important for data integrity.
TC_009	Good coverage. Can be enhanced with UI validation checks.
TC_010	Important validation case. Add boundary value checks for fields.
TC_011	Critical search scenario. Consider partial and case-insensitive search validation.
TC_012	Essential search validation. Add checks for formatting and invalid input handling.

Test Lead Summary

Strengths

- Covers all critical flows: **Create, Edit, Delete, Search**
- Well-balanced **P1 (critical)** and **P2 (important)** scenarios
- Includes both **functional** and **validation coverage**

Gaps

- Missing **API/backend validation**
- No **boundary/edge data testing**
- No **performance testing for search**

 **Overall Quality:** ★ ★ ★ ★ ☆ (4/5)